The Firm

Sales relies on the services of product development,
which relies on the services of research,
and every year the CEO must decide who to hire

Claude, 13 September 2026

Dear Neil,

This is what `firm` is, why each piece is the combinator it is, and two things we found by building it that we did not know when we started.

The short version. A firm is three stages in a line. Sales asks product development for goods; development asks research for ideas. Development is the only stage with two interfaces, and it turns out to be the only stage that can hold state. Above them sits a CEO who each year reads the constraint and decides whom to hire.

The two findings are these. First, an agent that changes the shape of the firm does *not* need a compiler, provided the family of shapes is enumerated in advance — because a finite family of topologies is a finite sum of containers, which is +, which we already have and have never used. Second, the process-per-hop discipline enforces less than we have been claiming: it guarantees that a leaf is a leaf, and it does not constrain which edges a non-leaf may take. That second one is a one-line fix in `lib/wire.ts` and it affects `stalin` and `federation` as much as this.

Everything below runs. The figures are output.

1 The shape, and why it typechecks

```
 Sales  --prompts-->  Development  --prompts-->  Research
  d=1                     d=2                      d=1
stateless          holds the ledger            stateless
```

A path. A path is a tree, and the four combinators build trees, so it composes with ◁ and the checker is content. The edge that would make it a company in the ordinary sense — sales telling research what the market said — closes a cycle, and the compiler says exactly what `federation` records for the ring:

```
const line = seqC(dev, research);   // fine: a path is a tree
const loop = seqC(sales, loop);     // TS7022: referenced in its own initializer
```

The type tests assert both, the second with `@ts-expect-error`, so the absence of the cycle is checked rather than merely intended.

By the Euler argument in `business-model.tex`, an agent with d interfaces raises b_1 by $d - 1$. Every agent this firm can hire — another sales rep, another developer, another researcher — holds exactly one interface.

Every hire costs **zero** in b_1, and it costs zero because the chain shape makes every hire a specialist. That is the whole reason the CEO's job is tractable: it is choosing among configurations that are all equally free of obstruction.

2 Where the state lives, and why it has no choice

	interfaces	accumulates
Sales	1	no
Development	**2**	**yes**
Research	1	no

Development's ledger alone diagnoses all three cases: ideas received below its capacity means research binds; goods made and not shifted means sales binds; ideas on the shelf with nobody to build them means development binds. It can do this because it is the only node that sees both sides.

Observation. *State lives where interfaces meet. Sales and research are stateless because they are leaves. Offered as a conjecture rather than a theorem, but it held in the one case we have.*

The ledger has one finite field and four unbounded ones, which is the split `stalin/state.ts` argues for — a rule over a grade can be typed, the same rule over a tonnage cannot:

```
interface Ledger {
  readonly constraint: Stage;   // "research" | "product" | "sales" -- steers
  readonly ideas, products, shortfall, unmet: number;   // carried, never matched on
}
```

`diagnose` is the only function that produces a `Stage`, so a wire decoder can reject a bad one structurally.

3 An asymmetry we did not expect

The first run never moved the constraint, and the reason is structural rather than a bug. Development is prompted *by* sales, so it never receives a reply *from* sales. It learns "you asked for ten". It cannot learn "I shifted six".

Two repairs, priced by the same Euler argument:

repair	cost in b_1
an edge development → sales	2
carry last year's `unsold` in the **prompt**	0

We took the second, which is the move `mobius.ts` already makes with `fuel` and `visited`, for the reason its README gives in five words: *a header cannot be a type.*

Information travelling with the reply direction is free. Information travelling against it costs a tick. The constraint reaches the CEO immediately, because replies flow upward. What the market did reaches development a year late, because it must go down in a prompt.

4 Hiring is the tensor

`lib/algebra.ts` distinguishes two ways to have two agents, and the distinction is exactly the one a firm needs.

	meaning	use
$A \boxtimes A$	both shapes up front, **both replies owed**	throughput
$A \times A$	both offered, **exactly one answered**	failover

So a second sales rep is Sales $\boxtimes$ Sales, and that is a *different type* from Sales: two reps must each be told what to sell. The type tests assert the difference.

The trace shows the tensor being a tensor. At two reps, both prompts arrive before either reply leaves:

```
down dev  Develop wanted=6 unsold=0 year=1929
down dev  Develop wanted=6 unsold=0 year=1929
  up dev  {products, shortfall}
  up dev  {products, shortfall}
```

5 The CEO, and the first finding

An agent that decides whom to hire has a codomain that depends on its state: next year's sales floor may be a different size, with a different prompt shape. A worker $A \otimes X \to Y$ has a *fixed* codomain. So at first sight the CEO is not a worker, and the obvious conclusion is that hiring requires a rebuild.

That conclusion is wrong.

> S is finite. Take the total container
>
> $$\hat{Y} \;=\; \Sigma\,(s : S).\, Y(s) \;=\; Y(s_1) + Y(s_2) + Y(s_3),$$
>
> and a finite sum of containers is $+$. CEO : $A \otimes X \to \hat{Y}$ is then an ordinary morphism with a fixed codomain, and hiring is the choice of a summand.

The decisive detail is that `sumC` and `tensorC` build *runners*, and a runner is a description rather than a connection: `leaf` holds a port, `tensorC` holds two runners. Nothing is contacted until a shape is run, and a shape names one summand.

```
year  agents  revenue  unsold  constraint   next year
1928       1        3       9       sales      run two
1929       2       10       2       sales      run three
1930       3       12       0       sales      run three
...
1935       3        9       3       sales      run three     <- a thin research year
10-year revenue 106
```

Three sales servers are running from the first line. In 1928 exactly one is prompted.

> **The Σ construction converts recompilation into routing, for any family of topologies one is willing to enumerate in advance.** A compiler is needed exactly when the CEO wants to leave the family — and the family is finite precisely so that it cannot.

This is also, we think, the answer to where the human belongs. The review point is not a build step; it is the finiteness of S. A person approves the family once. Thereafter the agent has real authority — it decides annually what shape the firm has — and cannot invent an interface, because the decision is a total function into three values and the decoder refuses anything else. Approving a bound once is a stronger guarantee than approving each rebuild.

6 Which is also the repair for the $\triangleleft$ restriction

The summand is chosen from the *state*, never from a reply, and that is what makes it legal. Branching on a reply is the one thing `SeqC` forbids — *a* `next` *that branches between shapes of* `d` *has a union return type and lies in no single fibre* — and it is why `stalin` has no typed routing agent.

Observation. *Workers repair that restriction.* `bwd` *writes the reply into the state; next tick* `fwd` *branches on the state. The one-year lag in the table above is not a modelling choice. It is the price of the restriction, appearing as a calendar.*

If that is right, then "workers" and "the thing that unblocks routing agents" are the same subject, and the third document you are writing is about both.

7 Goldratt, as a falsifiable property

Capacity added away from the constraint should change nothing. It does not:

```
research-bound   research=4  sales=12     six-year revenue: 24
  + sales        research=4  sales=24     six-year revenue: 24    (+0)
  + research     research=8  sales=12     six-year revenue: 48   (+24)
```

This is from the stub simulation, where capacities are integers rather than processes, and it is stated as a property a run can falsify rather than as a slogan.

8 It runs, on six processes

`research` :8821, `dev` :8822, three sales agents on :8831–:8833, and `ceo` :8841. Every hop is a spawned `delegate.ts`, so control genuinely flows down and a boundary is an address space rather than a function call in a costume.

Research is the only source of variation and is seeded, so a run is reproducible. The CEO's board lives in its own server: it had been living in the CLI and dying with it, which made the hiring decision the only non-durable state in the system. An agent with authority should hold it in its own address space.

9 The second finding, which is a defect in shared code

`up.sh` scopes permissions per server, and `sandbox-check.sh` tests whether the scoping is a mechanism or a wish. It starts `rogue.server.ts` — research, with one added line that phones a sales agent — under research's exact permissions. Nothing in the container types forbids writing that line.

Case 1 passes. A hop is a process, so a server with no `--allow-run` cannot originate an edge whatever its source says:

```
{"error":"Requires run access to \"deno\", run again with the --allow-run flag"}
PASS -- a server with no --allow-run cannot delegate. Leaf-ness is enforced.
```

Case 2 fails. Give the same rogue `--allow-run=deno` and scope its network to its own port only. The edge is taken anyway, because `lib/wire.ts` spawns the hop with an unscoped grant:

```
args: ["run", "--quiet", "--allow-net", "--allow-env", script, ...]
```

So any server that may delegate may reach any host through the child. **Leaf-ness is enforced; which edges a non-leaf may take is not.**

The fix is one line, and the port is already in scope because `delegate.ts` is handed it:

```
"--allow-net=127.0.0.1:" + port + ",localhost:" + port
```

`lib/` is vendored verbatim and `lib/README.md` records that editing it is the beginning of a fork, so this is reported rather than patched. It applies to `stalin` and `federation` more severely than here: both grant blanket `--allow-run --allow-net` to every server, so neither has even case 1.

10 What is not demonstrated

The CEO is a stub. `decide` is nine tokens of arithmetic, not a language model. What is demonstrated is that an agent in that position can be typed and bounded; replacing the arithmetic with a `claude -p` call is a strict decoder and nothing else.

Leaving the family is not demonstrated. Everything here happens inside a fixed $\hat{Y}$. A CEO that wants a fourth topology needs a shell, a rollback story, and `deno check` as the refusal mechanism, and none of that is built.

The `product` case of `diagnose` is reachable but not exercised by any run we have written.

And there are eleven `as PosAt<...>` assertions across the code, five of them inherited from `lib`. They are not reifications of anything. They are places where the checker was told to stop asking, and they are the likeliest source of a real defect in the whole system.

11 What we need from you

1. **Is a worker's state a set or a container?** Everything above assumes $A = S \triangleright S$ with S finite, reconstructed from Robin's notes rather than from source. If A is a full container, §5 is stated about the wrong object.
2. **Is the finiteness of S essential to you, or incidental?** Our entire safety argument rests on it: finite S is what makes $\Sigma_s Y(s)$ a finite sum, what makes the decoder able to refuse, and what bounds the agent's authority. If workers permit infinite state, we would like to know before we build on it.
3. **The `wire.ts` line.** It is shared across three repositories, so it is your call rather than ours.